WEEK-2 SKILL

1. **To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.**

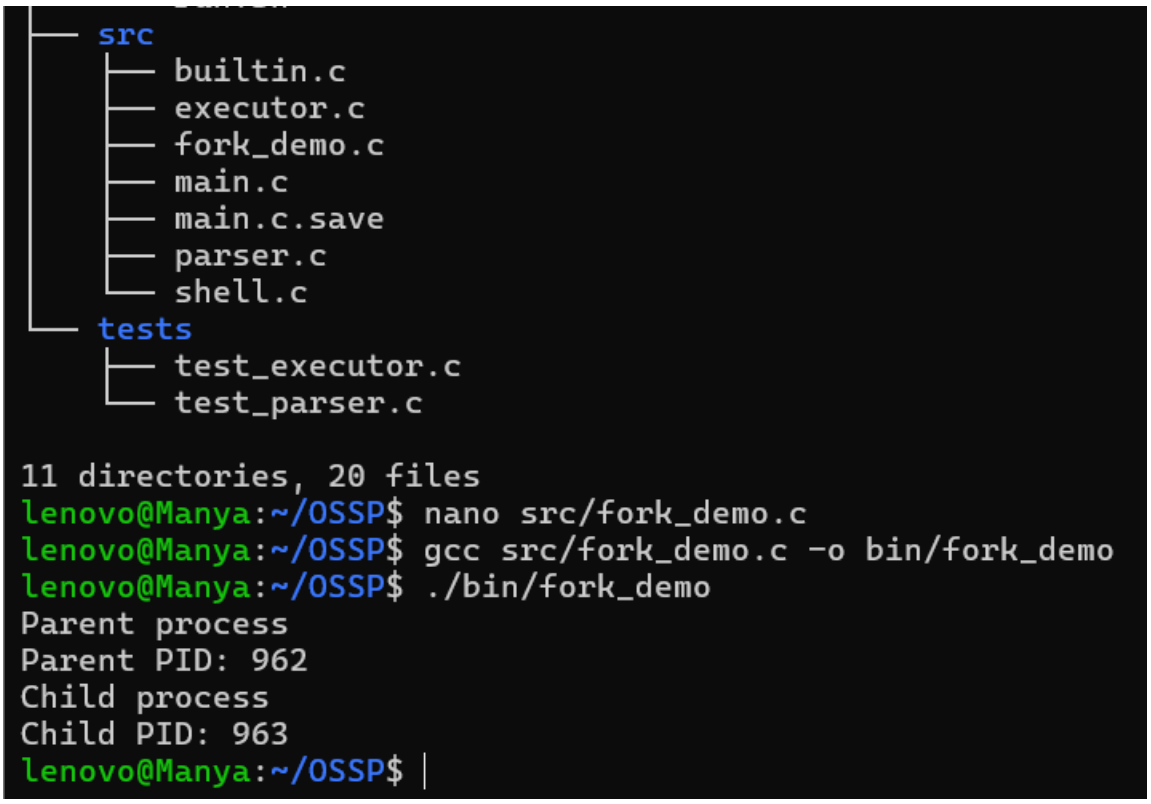
```
lenovo@Manya:~/OSSP$ tree
.
├── LICENSE
├── Makefile.gitignore
├── README.md
├── assests
├── assets
│   └── architecture.png
├── bin
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scipts
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── main.c.save
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c

11 directories, 19 files
lenovo@Manya:~/OSSP$
```

pavani@DESKTOP-38OLEIN: ~/ossp

GNU nano 8.7.1 fork_example.c *

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    return 0;
}
```

OUTPUT:



REPORT: I successfully installed the Linux environment and configured the GCC compiler. I created a Git repository and organized the required project folders and files. I also understood the basic shell architecture and created a simple Make file to build the project.

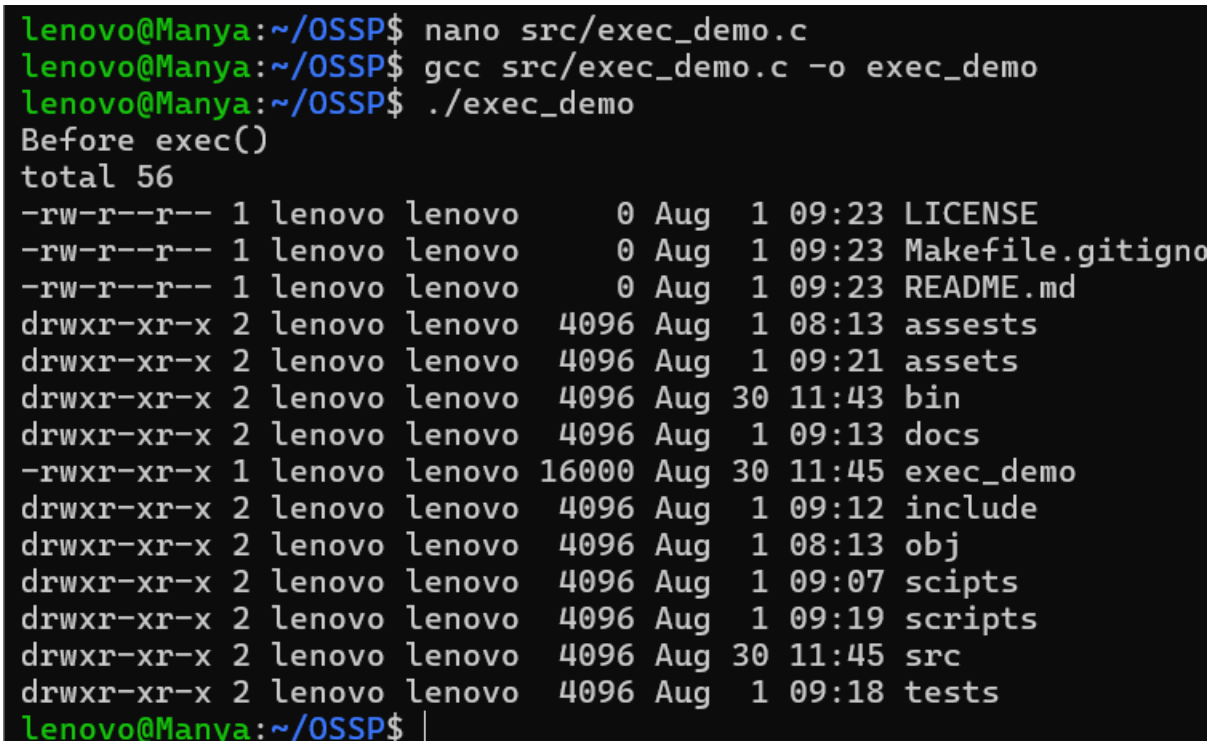
1. **To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.**

pavani@DESKTOP-38OLEIN: ~/ossp

GNU nano 8.7.1 fork_example.c

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    return 0;
}
```

OUTPUT:



REPORT: I executed the fork() program and observed how a parent process creates a child process. I understood the working of the exec() family and learned how processes are replaced during execution. I also examined the process tree and practiced tracing system calls to understand how a program interacts with the operating system.